A working-style demo: Codex + Claude Code

Building and auditing a string-compactification search

Edited from actual sessions. Prompt excerpts are verbatim; agent work is summarized.

I used Codex and Claude Code to build a research codebase for heterotic string compactifications with Standard-Model-like particle spectra. I set the scientific scope, validation requirements and research priorities; the agents implemented code, ran experiments and developed mathematical checks. The project began on a Mac mini M4 with 16 GB of memory.

1. Give Codex a specification with scientific acceptance criteria

> **My prompt:** "The first goal is correctness and reproducibility. Performance comes second."

My initial specification restricted the problem to five-line-bundle sums on favourable Calabi–Yau threefolds. It required primary-source checks, exact arithmetic, data provenance, a slow reference enumerator and a separate optimized search. Every filter had to state whether it was exact, necessary, sufficient or heuristic. An unsuccessful numerical search could not become a proof of absence.

Codex — work summary. Built the topology and bundle-algebra core, tests, CLI, checkpoints and certificate verification. It reproduced a published X_{7447} model's two indices, both -12, and matching second Chern classes. A sign inconsistency in a published formula was checked against another primary source, the Chern-class derivation and the known model before being used in a filter. The initial baseline passed 71 tests.

2. Hand the existing repository to Claude Code for a stronger audit

> **My prompt:** "The reference implementation must not reuse the optimized hierarchical pruning logic."

> **My prompt:** "Require exact set equality."

Once the first benchmark matched the published count of five models, I asked Claude Code to inspect the existing repository and continue from its validated state. I required an independent replay of all 43,046,721 ordered assignments in the smallest tetraquadric box. I also requested the actual published catalog, so the comparison would establish model-for-model agreement after the allowed permutations, rather than merely matching totals.

Claude Code — work summary. Implemented the independent replay and confirmed the same empty survivor set. It downloaded and parsed the catalog, then established exact orbit-set equality at the next benchmark. Switching tools was not itself an independent check; the separately implemented enumeration and explicit set comparison supplied that evidence.

3. Ask what a pruning rule actually proves

> **My prompt:** "A branch may be pruned only when the target is mathematically impossible for every completion inside the declared coefficient box."

My follow-up required a mathematical statement, proof or conjectural label, counterexample search, differential test and performance measurement. This led to an audit of the finite-coefficient slope test. Its rejection witnesses were sound, but passing the test did not establish stability. At the next charge bound, it retained an extra orbit. Claude Code implemented a complete version of that obstruction test, rejecting the extra orbit and restoring exact agreement with the seven published orbits in that box.

Steering the work beyond a successful benchmark

4. Change priorities when more scanning stops answering the main question

My prompt: "have you done phase 8 and 9?"

After the search and catalog comparisons worked, I redirected Claude Code toward the physical spectrum on quotient geometries. I asked it to compute the symmetry action on cohomology rather than rely on database lookups, including the vector-like sector needed to count Higgs pairs. The resulting work extended the code to equivariant cohomology and Wilson-line projections. This changed the next task from enlarging a charge box to testing what the surviving models actually predict.

5. Audit additional candidates before claiming a discovery

My prompt: "Do not call any candidate new until this audit is complete."

My prompt: "Do not infer a coupling merely from U(1) charge conservation."

When eleven additional orbits passed the implemented MSSM-spectrum conditions, I requested an immutable candidate manifest, comparison against the original scan's full conditions, and an explanation of each absence from its database. I then asked for geometric Yukawa calculations and dangerous-operator checks, with published models processed through the same pipeline as controls.

Claude Code — work summary. Implemented the audit and supported cohomological coupling calculations. The recorded ranking classified nine additional orbits as phenomenologically problematic and two as unknown under the stated perturbative criteria. None met the pipeline's strong-candidate requirements. The historical omissions were explained by restrictions of the earlier scan. A larger list of candidates was therefore not presented as eleven established new Standard Models.

6. Keep corrections visible and strengthen the tests

The sessions also contain corrections. During the Claude Code work, a cohomology comparison initially checked only single-degree cases and gave an overly broad agreement report. Checking every degree exposed apparent character-label disagreements. Further investigation traced these to applying an equivariant-structure shift on one side of the comparison but not the other. The comparison was corrected, the regression requirement strengthened, and the explanation appended to `RESULTS.md`.

A separate correction changed unsupported higher-level coupling calculations from "zero" to `UNKNOWN`. These distinctions matter: an uncomputed coupling and a mathematically vanishing coupling lead to different physical conclusions. My specification required the notebook to retain failures and unresolved cases, not just successful benchmarks.

What this demonstrates about how I work

I start with a precise research question and define what evidence would count as progress. I give agents room to implement and experiment, then use independent replay, catalog comparison and explicit mathematical assumptions to challenge their outputs. I redirect the work when the bottleneck changes, and keep promising candidates separate from verified claims. The accompanying repository contains the code, tests, benchmark artifacts, mathematical notes and research log needed to inspect this process.

Source trail. Initial Codex build session and master prompt; local Claude Code session *String landscape exhaustive replay and verification*; repository `RESULTS.md`, slope-obstruction notes and candidate-ranking artifacts.